

```

    "errors": [
      {
        "code": "None",
        "details": "string"
      }
    ],
    "warnings": [
      {
        "code": "ListingOptOutWarning",
        "details": "You have removed listing language(s): []"
      }
    ],
    "certificationReports": [
      {
      }
    ]
  },
  "fileUploadUrl":
    "https://productingestionbin1.blob.core.windows.net/ingestion/26920f66-b592-4439-9a9d-fb0f014902ec?sv=2014-02-14&sr=b&sig=usAN0kNFNnYE2tGQBI%2BARQWejX1Guiz7hdFtRhyK%2Bog%3D&se=2016-06-17T20:45:51Z&sp=rwl",
  "friendlyName": "Submission 2"
}

```

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
400	The submission could not be updated because the request is invalid.
409	The submission could not be updated because of the current state of the add-on, or the add-on uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Manage add-on submissions](#)
- [Get an add-on submission](#)
- [Create an add-on submission](#)
- [Commit an add-on submission](#)
- [Delete an add-on submission](#)
- [Get the status of an add-on submission](#)

Delete an add-on submission

Article • 10/20/2022

Use this method in the Microsoft Store submission API to delete an existing add-on (also known as in-app product or IAP) submission.

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

Method	Request URI
DELETE	<code>https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{inAppProductId}/submissions/{submissionId}</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <i><token></i> .

Request parameters

Name	Type	Description
inAppProductId	string	Required. The Store ID of the add-on that contains the submission to delete. The Store ID is available in Partner Center.
submissionId	string	Required. The ID of the submission to delete. This ID is available in the response data for requests to create an add-on submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to delete an add-on submission.

JSON

```
DELETE
https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/9NBLGGH4TNMP/submissions/115
2921504621230023 HTTP/1.1
Authorization: Bearer <your access token>
```

Response

If successful, this method returns an empty response body.

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
400	The request parameters are invalid.
404	The specified submission could not be found.
409	The specified submission was found but it could not be deleted in its current state, or the add-on uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Get an add-on submission](#)
- [Create an add-on submission](#)
- [Commit an add-on submission](#)
- [Update an add-on submission](#)
- [Get the status of an add-on submission](#)

Manage package flights

Article • 10/20/2022

Use the following methods in the Microsoft Store submission API to manage package flights for your apps. For an introduction to the Microsoft Store submission API, including prerequisites for using the API, see [Create and manage submissions using Microsoft Store services](#).

These methods can only be used to get, create, or delete package flights. To create submissions for package flights, see the methods in [Manage package flight submissions](#).

Method	URI	Description
GET	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}	Get a package flight
POST	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights	Create a package flight
DELETE	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}	Delete a package flight

Prerequisites

If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API before trying to use any of these methods.

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Manage package flight submissions](#)

Get a package flight

Article • 01/04/2023

Use this method in the Microsoft Store submission API to get data for a package flight for an app that is registered to your Partner Center account.

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

Method	Request URI
GET	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token>.

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight you want to get. The Store ID for the app is available in Partner Center.
flightId	string	Required. The ID of the package flight to get. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to retrieve information about a package flight with the ID 43e448df-97c9-4a43-a0bc-2a445e736bcd for an app with the Store ID value 9WZDNCRD91MD.

JSON

```
GET
https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd HTTP/1.1
Authorization: Bearer <your access token>
```

Response

The following example demonstrates the JSON response body for a successful call to this method. For more details about the values in the response body, see the following sections.

JSON

```
{
  "flightId": "43e448df-97c9-4a43-a0bc-2a445e736bcd",
  "friendlyName": "myflight",
  "lastPublishedFlightSubmission": {
    "id": "1152921504621086517",
    "resourceLocation": "flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621086517"
  },
  "pendingFlightSubmission": {
    "id": "115292150462124364",
    "resourceLocation": "flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243647"
  },
  "groupIds": [
    "0"
  ],
  "rankHigherThan": "671c2857-725e-4faf-9e9e-ea1191ef879c"
}
```

Response body

Value	Type	Description
-------	------	-------------

Value	Type	Description
flightId	string	The ID for the package flight. This value is supplied by Partner Center.
friendlyName	string	The name of the package flight, as specified by the developer.
lastPublishedFlightSubmission	object	An object that provides information about the last published submission for the package flight. For more information, see the Submission object section below.
pendingFlightSubmission	object	An object that provides information about the current pending submission for the package flight. For more information, see the Submission object section below.
groupIds	array	An array of strings that contain the IDs of the flight groups that are associated with the package flight. For more information about flight groups, see Package flights .
rankHigherThan	string	The friendly name of the package flight that is ranked immediately lower than the current package flight. For more information about ranking flight groups, see Package flights .

Submission object

The *lastPublishedFlightSubmission* and *pendingFlightSubmission* values in the response body contain objects that provide resource information about a submission for the package flight. These objects have the following values.

Value	Type	Description
id	string	The ID of the submission.
resourceLocation	string	A relative path that you can append to the base <code>https://manage.devcenter.microsoft.com/v1.0/my/</code> request URI to retrieve the complete data for the submission.

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
400	The request is invalid.
404	The specified package flight could not be found.

Error code	Description
409	The app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Create a package flight](#)
- [Delete a package flight](#)

Create a package flight

Article • 11/01/2022

Use this method in the Microsoft Store submission API to create a package flight for an app that is registered to your Partner Center account.

ⓘ Note

This method creates a package flight without any submissions. To create a submission for package flight, see the methods in [Manage package flight submissions](#).

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

Method	Request URI
POST	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token>.

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app for which you want to create a package flight. For more information about the Store ID, see View app identity details .

Request body

The request body has the following parameters.

Parameter	Type	Description	Required
friendlyName	string	The name of the package flight, as specified by the developer.	No
groupIds	array	An array of strings that contain the IDs of the flight groups that are associated with the package flight. For more information about flight groups, see Package flights .	No
rankHigherThan	string	The friendly name of the package flight that is ranked immediately lower than the current package flight. If you do not set this parameter, the new package flight will have the highest rank of all package flights. For more information about ranking flight groups, see Package flights .	No

Request example

The following example demonstrates how to create a new package flight for an app that has the Store ID 9WZDNCRD911W.

syntax

```
POST
https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights HTTP/1.1
Authorization: Bearer eyJ0eXAiOiJKV1Q...
Content-Type: application/json
{
  "friendlyName": "myflight",
  "groupIds": [
    0
  ],
  "rankHigherThan": null
}
```

Response

The following example demonstrates the JSON response body for a successful call to this method. For more details about the values in the response body, see the following sections.

JSON

```
{
  "flightId": "43e448df-97c9-4a43-a0bc-2a445e736bcd",
  "friendlyName": "myflight",
  "groupIds": [
    "0"
  ],
  "rankHigherThan": "671c2857-725e-4faf-9e9e-ea1191ef879c"
}
```

Response body

Value	Type	Description
flightId	string	The ID for the package flight. This value is supplied by Partner Center.
friendlyName	string	The name of the package flight, as specified in the request.
groupIds	array	An array of strings that contain the IDs of the flight groups that are associated with the package flight, as specified in the request. For more information about flight groups, see Package flights .
rankHigherThan	string	The friendly name of the package flight that is ranked immediately lower than the current package flight, as specified in the request. For more information about ranking flight groups, see Package flights .

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
400	The request is invalid.
409	The package flight could not be created because of its current state, or the app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Get a package flight](#)
- [Delete a package flight](#)

Delete a package flight

Article • 10/20/2022

Use this method in the Microsoft Store submission API to delete a package flight for an app that is registered to your Partner Center account.

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

Method	Request URI
DELETE	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token>.

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight you want to delete. The Store ID for the app is available in Partner Center.
flightId	string	Required. The ID of the package flight to delete. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to delete a package flight.

JSON

```
DELETE
https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd HTTP/1.1
Authorization: Bearer <your access token>
```

Response

If successful, this method returns an empty response body.

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
400	The request parameters are invalid.
404	The specified package flight could not be found.
409	The specified package flight was found but it could not be deleted in its current state, or the app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Create a package flight](#)
- [Get a package flight](#)

Manage package flight submissions

Article • 03/16/2023

The Microsoft Store submission API provides methods you can use to manage package flight submissions for your apps, including gradual package rollouts. For an introduction to the Microsoft Store submission API, including prerequisites for using the API, see [Create and manage submissions using Microsoft Store services](#).

 Important

If you use the Microsoft Store submission API to create a submission for a package flight, be sure to make further changes to the submission only by using the API, rather than Partner Center. If you use the dashboard to change a submission that you originally created by using the API, you will no longer be able to change or commit that submission by using the API. In some cases, the submission could be left in an error state where it cannot proceed in the submission process. If this occurs, you must delete the submission and create a new submission.

Methods for managing package flight submissions

Use the following methods to get, create, update, commit, or delete a package flight submission. Before you can use these methods, the package flight must already exist in Partner Center. You can create a package flight [in Partner Center](#) or by using the Microsoft Store submission API methods in described in [Manage package flights](#).

Method	URI	Description
GET	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}	Get an existing package flight submission
GET	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/status	Get the status of an existing package flight submission
POST	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions	Create a new package flight submission
PUT	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}	Update an existing package flight submission
POST	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/commit	Commit a new or updated package flight submission
DELETE	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}	Delete a package flight submission

Create a package flight submission

To create a submission for a package flight, follow this process.

1. If you have not yet done so, complete the prerequisites described in [Create and manage submissions using Microsoft Store services](#), including associating an Azure AD application with your Partner Center account and obtaining your client ID and key. You only need

to do this one time; after you have the client ID and key, you can reuse them any time you need to create a new Azure AD access token.

2. [Obtain an Azure AD access token](#). You must pass this access token to the methods in the Microsoft Store submission API. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
3. [Create a package flight submission](#) by executing the following method in the Microsoft Store submission API. This method creates a new in-progress submission, which is a copy of your last published submission.

JSON

```
POST https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions
```

The response body contains a [flight submission](#) resource that includes the ID of the new submission, the shared access signature (SAS) URI for uploading any packages for the submission to Azure Blob Storage, and the data for the new submission (including all the listings and pricing information).

Note

A SAS URI provides access to a secure resource in Azure storage without requiring account keys. For background information about SAS URIs and their use with Azure Blob Storage, see [Shared Access Signatures, Part 1: Understanding the SAS model](#) and [Shared Access Signatures, Part 2: Create and use a SAS with Blob storage](#).

4. If you are adding new packages for the submission, [prepare the packages](#) and add them to a ZIP archive.
5. Revise the [flight submission](#) data with any required changes for the new submission, and execute the following method to [update the package flight submission](#).

JSON

```
PUT https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}
```

Note

If you are adding new packages for the submission, make sure you update the submission data to refer to the name and relative path of these files in the ZIP archive.

6. If you are adding new packages for the submission, upload the ZIP archive to [Azure Blob Storage](#) using the SAS URI that was provided in the response body of the POST method you called earlier. There are different Azure libraries you can use to do this on a variety of platforms, including:

- [Azure Storage Client Library for .NET](#)
- [Azure Storage SDK for Java](#)
- [Azure Storage SDK for Python](#)

The following C# code example demonstrates how to upload a ZIP archive to Azure Blob Storage using the [CloudBlockBlob](#) class in the Azure Storage Client Library for .NET. This example assumes that the ZIP archive has already been written to a stream object.

C#

```
string sasUrl = "https://productingestionbin1.blob.core.windows.net/ingestion/26920f66-b592-4439-9a9d-fb0f014902ec?sv=2014-02-14&sr=b&sig=usAN0kNFNnYE2tGQBI%2BARQWejX1Guiz7hdFtRhyK%2Bog%3D&se=2016-06-17T20:45:51Z&sp=rw1";
Microsoft.WindowsAzure.Storage.Blob.CloudBlockBlob blockBob =
    new Microsoft.WindowsAzure.Storage.Blob.CloudBlockBlob(new System.Uri(sasUrl));
await blockBob.UploadFromStreamAsync(stream);
```

7. [Commit the package flight submission](#) by executing the following method. This will alert Partner Center that you are done with your submission and that your updates should now be applied to your account.

JSON

```
POST https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/commit
```


8. Check on the commit status by executing the following method to [get the status of the package flight submission](#).

```
JSON

GET
https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/status
```

To confirm the submission status, review the *status* value in the response body. This value should change from **CommitStarted** to either **PreProcessing** if the request succeeds or to **CommitFailed** if there are errors in the request. If there are errors, the *statusDetails* field contains further details about the error.

9. After the commit has successfully completed, the submission is sent to the Store for ingestion. You can continue to monitor the submission progress by using the previous method, or by visiting Partner Center.

Code examples

The following articles provide detailed code examples that demonstrate how to create a package flight submission in several different programming languages:

- [C# code examples](#)
- [Java code examples](#)
- [Python code examples](#)

StoreBroker PowerShell module

As an alternative to calling the Microsoft Store submission API directly, we also provide an open-source PowerShell module which implements a command-line interface on top of the API. This module is called [StoreBroker](#). You can use this module to manage your app, flight, and add-on submissions from the command line instead of calling the Microsoft Store submission API directly, or you can simply browse the source to see more examples for how to call this API. The StoreBroker module is actively used within Microsoft as the primary way that many first-party applications are submitted to the Store.

For more information, see our [StoreBroker page on GitHub](#).

Manage a gradual package rollout for a package flight submission

You can gradually roll out the updated packages in a package flight submission to a percentage of your app's customers on Windows 10 and Windows 11. This allows you to monitor feedback and analytic data for the specific packages to make sure you're confident about the update before rolling it out more broadly. You can change the rollout percentage (or halt the update) for a published submission without having to create a new submission. For more details, including instructions for how to enable and manage a gradual package rollout in Partner Center, see [this article](#).

To programmatically enable a gradual package rollout for a package flight submission, follow this process using methods in the Microsoft Store submission API:

1. [Create a package flight submission](#) or [get a package flight submission](#).
2. In the response data, locate the [packageRollout](#) resource, set the *isPackageRollout* field to true, and set the *packageRolloutPercentage* field to the percentage of your app's customers who should get the updated packages.
3. Pass the updated package flight submission data to the [update a package flight submission](#) method.

After a gradual package rollout is enabled for a package flight submission, you can use the following methods to programmatically get, update, halt, or finalize the gradual rollout.

Method	URI
GET	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/packagerollout

Method	URI
POST	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/updatepackagerolloutpercentag
POST	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/haltpackagerollout
POST	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/finalizepackagerollout

Data resources

The Microsoft Store submission API methods for managing package flight submissions use the following JSON data resources.

Flight submission resource

This resource describes a package flight submission.

```
JSON

{
  "id": "1152921504621243649",
  "flightId": "cd2e368a-0da5-4026-9f34-0e7934bc6f23",
  "status": "PendingCommit",
  "statusDetails": {
    "errors": [],
    "warnings": [],
    "certificationReports": []
  },
  "flightPackages": [
    {
      "fileName": "newPackage.appx",
      "fileStatus": "PendingUpload",
      "id": "",
      "version": "1.0.0.0",
      "languages": ["en-us"],
      "capabilities": [],
      "minimumDirectXVersion": "None",
      "minimumSystemRam": "None"
    }
  ],
  "packageDeliveryOptions": {
    "packageRollout": {
      "isPackageRollout": false,
      "packageRolloutPercentage": 0.0,
      "packageRolloutStatus": "PackageRolloutNotStarted",
      "fallbackSubmissionId": ""
    },
    "isMandatoryUpdate": false,
    "mandatoryUpdateEffectiveDate": "1601-01-01T00:00:00.000000Z"
  },
  "fileUploadUrl": "https://productingestionbin1.blob.core.windows.net/ingestion/8b389577-5d5e-4cbe-a744-1ff2e97a9eb8?sv=2014-02-14&sr=b&sig=wgMCQPjPDkuuxNLkeG35rfHaMToebCxBNMPw7WABdXU%3D&se=2016-06-17T21:29:44Z&sp=rwl",
  "targetPublishMode": "Immediate",
  "targetPublishDate": "",
  "notesForCertification": "No special steps are required for certification of this app."
}
```

This resource has the following values.

Value	Type	Description
id	string	The ID for the submission.
flightId	string	The ID of the package flight that the submission is associated with.
status	string	The status of the submission. This can be one of the following values: • None • Canceled • PendingCommit • CommitStarted • CommitFailed • PendingPublication • Publishing • Published • PublishFailed • PreProcessing • PreProcessingFailed • Certification • CertificationFailed • Release • ReleaseFailed
statusDetails	object	A status details resource that contains additional details about the status of the submission, including information about any errors.
flightPackages	array	Contains flight package resources that provide details about each package in the submission.
packageDeliveryOptions	object	A package delivery options resource that contains gradual package rollout and mandatory update settings for the submission.
fileUploadUrl	string	The shared access signature (SAS) URI for uploading any packages for the submission. If you are adding new packages for the submission, upload the ZIP archive that contains the packages to this URI. For more information, see Create a package flight submission .
targetPublishMode	string	The publish mode for the submission. This can be one of the following values: • Immediate • Manual • SpecificDate
targetPublishDate	string	The publish date for the submission in ISO 8601 format, if the <i>targetPublishMode</i> is set to SpecificDate.
notesForCertification	string	Provides additional info for the certification testers, such as test account credentials and steps to access and verify features. For more information, see Notes for certification .

Status details resource

This resource contains additional details about the status of a submission. This resource has the following values.

Value	Type	Description
errors	object	An array of status detail resources that contain error details for the submission.
warnings	object	An array of status detail resources that contain warning details for the submission.
certificationReports	object	An array of certification report resources that provide access to the certification report data for the submission. You can examine these reports for more information if the certification fails.

Status detail resource

This resource contains additional information about any related errors or warnings for a submission. This resource has the following values.

Value	Type	Description
code	string	A submission status code that describes the type of error or warning.
details	string	A message with more details about the issue.

Certification report resource

This resource provides access to the certification report data for a submission. This resource has the following values.

Value	Type	Description
date	string	The date and time the report was generated, in ISO 8601 format.
reportUrl	string	The URL at which you can access the report.

Flight package resource

This resource provides details about a package in a submission.

```
JSON

{
  "flightPackages": [
    {
      "fileName": "newPackage.appx",
      "fileStatus": "PendingUpload",
      "id": "",
      "version": "1.0.0.0",
      "languages": ["en-us"],
      "capabilities": [],
      "minimumDirectXVersion": "None",
      "minimumSystemRam": "None"
    }
  ],
}
```

This resource has the following values.

 **Note**

When calling the **update a package flight submission** method, only the *fileName*, *fileStatus*, *minimumDirectXVersion*, and *minimumSystemRam* values of this object are required in the request body. The other values are populated by Partner Center.

Value	Type	Description
fileName	string	The name of the package.
fileStatus	string	The status of the package. This can be one of the following values: - None - PendingUpload - Uploaded - PendingDelete
id	string	An ID that uniquely identifies the package. This value is used by Partner Center.
version	string	The version of the app package. For more information, see Package version numbering .
architecture	string	The architecture of the app package (for example, ARM).
languages	array	An array of language codes for the languages the app supports. For more information, see Supported languages .
capabilities	array	An array of capabilities required by the package. For more information about capabilities, see App capability declarations .
minimumDirectXVersion	string	The minimum DirectX version that is supported by the app package. This can be set only for apps that target Windows 8.x; it is ignored for apps that target other versions. This can be one of the following values: - None - DirectX93 - DirectX100
minimumSystemRam	string	The minimum RAM that is required by the app package. This can be set only for apps that target Windows 8.x; it is ignored for apps that target other versions. This can be one of the following values: - None - Memory2GB

Package delivery options resource

This resource contains gradual package rollout and mandatory update settings for the submission.

JSON

```
{
  "packageDeliveryOptions": {
    "packageRollout": {
      "isPackageRollout": false,
      "packageRolloutPercentage": 0.0,
      "packageRolloutStatus": "PackageRolloutNotStarted",
      "fallbackSubmissionId": "0"
    },
    "isMandatoryUpdate": false,
    "mandatoryUpdateEffectiveDate": "1601-01-01T00:00:00.0000000Z"
  },
}
```

This resource has the following values.

Value	Type	Description
packageRollout	object	A package rollout resource that contains gradual package rollout settings for the submission.
isMandatoryUpdate	boolean	Indicates whether you want to treat the packages in this submission as mandatory for self-installing app updates. For more information about mandatory packages for self-installing app updates, see Download and install package updates for your app .
mandatoryUpdateEffectiveDate	date	The date and time when the packages in this submission become mandatory, in ISO 8601 format and UTC time zone.

Package rollout resource

This resource contains gradual [package rollout settings](#) for the submission. This resource has the following values.

Value	Type	Description
isPackageRollout	boolean	Indicates whether gradual package rollout is enabled for the submission.
packageRolloutPercentage	float	The percentage of users who will receive the packages in the gradual rollout.
packageRolloutStatus	string	One of the following strings that indicates the status of the gradual package rollout: - PackageRolloutNotStarted - PackageRolloutInProgress - PackageRolloutComplete - PackageRolloutStopped
fallbackSubmissionId	string	The ID of the submission that will be received by customers who do not get the gradual rollout packages.

Note

The *packageRolloutStatus* and *fallbackSubmissionId* values are assigned by Partner Center, and are not intended to be set by the developer. If you include these values in a request body, these values will be ignored.

Enums

These methods use the following enums.

Submission status code

The following codes represent the status of a submission.

Code	Description
None	No code was specified.
InvalidArchive	The ZIP archive containing the package is invalid or has an unrecognized archive format.
MissingFiles	The ZIP archive does not have all files which were listed in your submission data, or they are in the wrong location in the archive.
PackageValidationFailed	One or more packages in your submission failed to validate.

Code	Description
InvalidParameterValue	One of the parameters in the request body is invalid.
InvalidOperation	The operation you attempted is invalid.
InvalidState	The operation you attempted is not valid for the current state of the package flight.
ResourceNotFound	The specified package flight could not be found.
ServiceError	An internal service error prevented the request from succeeding. Try the request again.
ListingOptOutWarning	The developer removed a listing from a previous submission, or did not include listing information that is supported by the package.
ListingOptInWarning	The developer added a listing.
UpdateOnlyWarning	The developer is trying to insert something that only has update support.
Other	The submission is in an unrecognized or uncategorized state.
PackageValidationWarning	The package validation process resulted in a warning.

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Manage package flights using the Microsoft Store submission API](#)
- [Get a package flight submission](#)
- [Create a package flight submission](#)
- [Update a package flight submission](#)
- [Commit a package flight submission](#)
- [Delete a package flight submission](#)
- [Get the status of a package flight submission](#)

Get a package flight submission

Article • 11/01/2022

Use this method in the Microsoft Store submission API to get data for an existing package flight submission. For more information about the process of creating a package flight submission by using the Microsoft Store submission API, see [Manage package flight submissions](#).

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
- Create a package flight submission for an app in Partner Center. You can do this in Partner Center, or you can do this by using the [create a package flight submission](#) method.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

Method	Request URI
GET	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions{submissionId}</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token>.

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight submission you want to get. For more information about the Store ID, see View app identity details .
flightId	string	Required. The ID of the package flight that contains the submission you want to get. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.
submissionId	string	Required. The ID of the submission to get. This ID is available in the response data for requests to create a package flight submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to get a package flight submission for an app that has the Store ID 9WZDNCRD91MD.

JSON

```
POST https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243649 HTTP/1.1
Authorization: Bearer <your access token>
```

Response

The following example demonstrates the JSON response body for a successful call to this method. The response body contains information about the specified submission. For more details about the values in the response body, see [Package flight submission resource](#).

JSON

```
{
  "id": "1152921504621243649",
  "flightId": "cd2e368a-0da5-4026-9f34-0e7934bc6f23",
  "status": "PendingCommit",
  "statusDetails": {
    "errors": [],
    "warnings": [],
    "certificationReports": []
  },
  "flightPackages": [
    {
      "fileName": "newPackage.appx",
      "fileStatus": "PendingUpload",
      "id": "",
      "version": "1.0.0.0",
      "languages": ["en-us"],
      "capabilities": [],
      "minimumDirectXVersion": "None",
      "minimumSystemRam": "None"
    }
  ],
  "packageDeliveryOptions": {
    "packageRollout": {
      "isPackageRollout": false,
      "packageRolloutPercentage": 0.0,
      "packageRolloutStatus": "PackageRolloutNotStarted",
      "fallbackSubmissionId": "0"
    },
    "isMandatoryUpdate": false,
    "mandatoryUpdateEffectiveDate": "1601-01-01T00:00:00.0000000Z"
  },
  "fileUploadUrl": "https://productingestionbin1.blob.core.windows.net/ingestion/8b389577-5d5e-4cbe-a744-1ff2e97a9eb8?sv=2014-02-14&sr=b&sig=wgMCQPjPDkuuxNLkeG35rfHaMTuebCxBNMPw7WABdXU%3D&se=2016-06-17T21:29:44Z&sp=rwl",
  "targetPublishMode": "Immediate",
  "targetPublishDate": "",
  "notesForCertification": "No special steps are required for certification of this app."
}
```

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
------------	-------------

Error code	Description
404	The package flight submission could not be found.
409	The package flight submission does not belong to the specified package flight, or the app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Manage package flight submissions](#)
- [Create a package flight submission](#)
- [Commit a package flight submission](#)
- [Update a package flight submission](#)
- [Delete a package flight submission](#)

Get the status of a package flight submission

Article • 11/01/2022

Use this method in the Microsoft Store submission API to get the status of a package flight submission. For more information about the process of creating a package flight submission by using the Microsoft Store submission API, see [Manage package flight submissions](#).

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
- Create a package flight submission for one of your apps. You can do this in Partner Center, or you can do this by using the [create a package flight submission](#) method.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

Method	Request URI
GET	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/status</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token>.

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight submission for which you want to get the status. For more information about the Store ID, see View app identity details .
flightId	string	Required. The ID of the package flight that contains the submission for which you want to get the status. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.
submissionId	string	Required. The ID of the submission for which you want to get the status. This ID is available in the response data for requests to create a package flight submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to get the status of a package flight submission.

JSON

```
GET https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243649/status HTTP/1.1
Authorization: Bearer <your access token>
```

Response

The following example demonstrates the JSON response body for a successful call to this method. The response body contains information about the specified submission. For more details about the values in the response body, see the following section.

JSON

```
{
  "status": "PendingCommit",
  "statusDetails": {
    "errors": [],
    "warnings": [],
    "certificationReports": []
  },
}
```

Response body

Value	Type	Description
status	string	The status of the submission. This can be one of the following values: • None • Canceled • PendingCommit • CommitStarted • CommitFailed • PendingPublication • Publishing • Published • PublishFailed • PreProcessing • PreProcessingFailed • Certification • CertificationFailed • Release • ReleaseFailed
statusDetails	object	Contains additional details about the status of the submission, including information about any errors. For more information, see Status details resource .

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
404	The submission could not be found.
409	The app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Manage package flight submissions](#)

- [Get an app submission](#)
- [Create an app submission](#)
- [Commit an app submission](#)
- [Update an app submission](#)
- [Delete an app submission](#)

Create a package flight submission

Article • 11/01/2022

Use this method in the Microsoft Store submission API to create a new submission for a package flight for an app. After you successfully create a new submission by using this method, [update the submission](#) to make any necessary changes to the submission data, and then [commit the submission](#) for ingestion and publishing.

For more information about how this method fits into the process of creating a package flight submission by using the Microsoft Store submission API, see [Manage package flight submissions](#).

ⓘ Note

This method creates a submission for an existing package flight. To create a package flight, use the [create a package flight](#) method.

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
- Create a package flight for an app. You can do this in Partner Center, or you can do this by using the [create a package flight](#) method.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

Method	Request URI
POST	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token>.

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app for which you want to create a package flight submission. For more information about the Store ID, see View app identity details .
flightId	string	Required. The ID of the package flight for which you want to add the submission. This ID is available in the response data for requests to create a package flight and get package flights for an app .

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to create a new package flight submission for an app that has the Store ID 9WZDNCRD91MD.

JSON

```
POST
https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions HTTP/1.1
Authorization: Bearer <your access token>
```

Response

The following example demonstrates the JSON response body for a successful call to this method. The response body contains information about the new submission. For more details about the values in the response body, see [Package flight submission resource](#).

JSON

```
{
  "id": "1152921504621243649",
  "flightId": "cd2e368a-0da5-4026-9f34-0e7934bc6f23",
  "status": "PendingCommit",
  "statusDetails": {
    "errors": [],
    "warnings": [],
    "certificationReports": []
  },
  "flightPackages": [
    {
      "fileName": "newPackage.appx",
      "fileStatus": "PendingUpload",
      "id": "",
      "version": "1.0.0.0",
      "languages": ["en-us"],
      "capabilities": [],
      "minimumDirectXVersion": "None",
      "minimumSystemRam": "None"
    }
  ]
}
```

```
"packageDeliveryOptions": {
  "packageRollout": {
    "isPackageRollout": false,
    "packageRolloutPercentage": 0.0,
    "packageRolloutStatus": "PackageRolloutNotStarted",
    "fallbackSubmissionId": "0"
  },
  "isMandatoryUpdate": false,
  "mandatoryUpdateEffectiveDate": "1601-01-01T00:00:00.0000000Z"
},
"fileUploadUrl": "https://productingestionbin1.blob.core.windows.net/ingestion/8b389577-5d5e-4cbe-a744-1ff2e97a9eb8?sv=2014-02-14&sr=b&sig=wgMCQPjPDkuuxNLkeG35rfHaMToebCxBNMPw7WABdXU%3D&se=2016-06-17T21:29:44Z&sp=rwl",
"targetPublishMode": "Immediate",
"targetPublishDate": "",
"notesForCertification": "No special steps are required for certification of this app."
}
```

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
400	The package flight submission could not be created because the request is invalid.
409	The package flight submission could not be created because of the current state of the app, or the app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Manage package flight submissions](#)
- [Get a package flight submission](#)
- [Commit a package flight submission](#)
- [Update a package flight submission](#)
- [Delete a package flight submission](#)
- [Get the status of a package flight submission](#)

Commit a package flight submission

Article • 10/20/2022

Use this method in the Microsoft Store submission API to commit a new or updated package flight submission to Partner Center. The commit action alerts Partner Center that the submission data has been uploaded (including any related packages). In response, Partner Center commits the changes to the submission data for ingestion and publishing. After the commit operation succeeds, the changes to the submission are shown in Partner Center.

For more information about how the commit operation fits into the process of creating a package flight submission by using the Microsoft Store submission API, see [Manage package flight submissions](#).

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
- [Create a package flight submission](#) and then [update the submission](#) with any necessary changes to the submission data.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

Method	Request URI
POST	https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/commit

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token>.

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight submission you want to commit. The Store ID for the app is available in Partner Center.
flightId	string	Required. The ID of the package flight that contains the submission to commit. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.
submissionId	string	Required. The ID of the submission to commit. This ID is available in the response data for requests to create a package flight submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to commit a package flight submission.

JSON

```
POST https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243649/commit HTTP/1.1
Authorization: Bearer <your access token>
```

Response

The following example demonstrates the JSON response body for a successful call to this method. For more details about the values in the response body, see the following sections.

JSON

```
{
  "status": "CommitStarted"
}
```

Response body

Value	Type	Description
status	string	The status of the submission. This can be one of the following values: - None - Canceled - PendingCommit - CommitStarted - CommitFailed - PendingPublication - Publishing - Published - PublishFailed - PreProcessing - PreProcessingFailed - Certification - CertificationFailed - Release - ReleaseFailed

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
400	The request parameters are invalid.
404	The specified submission could not be found.
409	The specified submission was found but it could not be committed in its current state, or the app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Manage package flight submissions](#)
- [Get a package flight submission](#)

- [Create a package flight submission](#)
- [Update a package flight submission](#)
- [Delete a package flight submission](#)
- [Get the status of a package flight submission](#)

Update a package flight submission

Article • 10/17/2024

Use this method in the Microsoft Store submission API to update an existing package flight submission. After you successfully update a submission by using this method, you must [commit the submission](#) for ingestion and publishing.

For more information about how this method fits into the process of creating a package flight submission by using the Microsoft Store submission API, see [Manage package flight submissions](#).

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
- Create a package flight submission for one of your apps. You can do this in Partner Center, or you can do this by using the [create a package flight submission](#) method.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

 Expand table

Method	Request URI
PUT	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}</code>

Request header

 Expand table

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <i><token></i> .

Request parameters

 Expand table

Name	Type	Description
applicationId	string	Required. The Store ID of the app for which you want to update a package flight submission. For more information about the Store ID, see View app identity details .
flightId	string	Required. The ID of the package flight for which you want to update a submission. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.
submissionId	string	Required. The ID of the submission to update. This ID is available in the response data for requests to create a package flight submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.

Name	Type	Description
packageId	string	Required. The ID of the existing Package.

Request body

The request body has the following parameters.

 Expand table

Value	Type	Description
flightPackages	array	Contains objects that provide details about each package in the submission. For more details about the values in the response body, see Flight package resource . When calling this method to update an app submission, only the <i>fileName</i> , <i>fileStatus</i> , <i>minimumDirectXVersion</i> , and <i>minimumSystemRam</i> values of these objects are required in the request body. The other values are populated by Partner Center.
packageDeliveryOptions	object	Contains gradual package rollout and mandatory update settings for the submission. For more information, see Package delivery options object .
targetPublishMode	string	The publish mode for the submission. This can be one of the following values: - Immediate - Manual - SpecificDate
targetPublishDate	string	The publish date for the submission in ISO 8601 format, if the <i>targetPublishMode</i> is set to SpecificDate.
notesForCertification	string	Provides additional info for the certification testers, such as test account credentials and steps to access and verify features. For more information, see Notes for certification .

Request example

The following example demonstrates how to update a package flight submission for an app.

JSON

```
PUT https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243649 HTTP/1.1
Authorization: Bearer <your access token>
Content-Type: application/json
{
  "flightPackages": [
    {
      "id": "existingPackage.Id",
      "fileName": "newPackage.appx",
      "fileStatus": "PendingUpload",
      "minimumDirectXVersion": "None",
      "minimumSystemRam": "None"
    }
  ],
  "packageDeliveryOptions": {
    "packageRollout": {
      "isPackageRollout": false,
      "packageRolloutPercentage": 0.0,
      "packageRolloutStatus": "PackageRolloutNotStarted",
      "fallbackSubmissionId": "0"
    },
    "isMandatoryUpdate": false,
    "mandatoryUpdateEffectiveDate": "1601-01-01T00:00:00.0000000Z"
  },
  "targetPublishMode": "Immediate",
}
```

```
"targetPublishDate": "",
"notesForCertification": "No special steps are required for certification of this app."
}
```

Response

The following example demonstrates the JSON response body for a successful call to this method. The response body contains information about the updated submission. For more details about the values in the response body, see [Package flight submission resource](#).

JSON

```
{
  "id": "1152921504621243649",
  "flightId": "cd2e368a-0da5-4026-9f34-0e7934bc6f23",
  "status": "PendingCommit",
  "statusDetails": {
    "errors": [],
    "warnings": [],
    "certificationReports": []
  },
  "flightPackages": [
    {
      "fileName": "newPackage.appx",
      "fileStatus": "PendingUpload",
      "id": "",
      "version": "1.0.0.0",
      "languages": ["en-us"],
      "capabilities": [],
      "minimumDirectXVersion": "None",
      "minimumSystemRam": "None"
    }
  ],
  "packageDeliveryOptions": {
    "packageRollout": {
      "isPackageRollout": false,
      "packageRolloutPercentage": 0.0,
      "packageRolloutStatus": "PackageRolloutNotStarted",
      "fallbackSubmissionId": "0"
    },
    "isMandatoryUpdate": false,
    "mandatoryUpdateEffectiveDate": "1601-01-01T00:00:00.000000Z"
  },
  "fileUploadUrl": "https://productingestionbin1.blob.core.windows.net/ingestion/8b389577-5d5e-4cbe-a744-1ff2e97a9eb8?sv=2014-02-14&sr=b&sig=wgMCQPjPDkuuxNLkeG35rfHaMToebCxBNMPw7WABdXU%3D&se=2016-06-17T21:29:44Z&sp=rwl",
  "targetPublishMode": "Immediate",
  "targetPublishDate": "",
  "notesForCertification": "No special steps are required for certification of this app."
}
```

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

 Expand table

Error code	Description
400	The package flight submission could not be updated because the request is invalid.

Error code	Description
409	The package flight submission could not be updated because of the current state of the app, or the app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Manage package flight submissions](#)
- [Get a package flight submission](#)
- [Create a package flight submission](#)
- [Commit a package flight submission](#)
- [Delete a package flight submission](#)
- [Get the status of a package flight submission](#)

Feedback

Was this page helpful?



[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Delete a package flight submission

Article • 11/01/2022

Use this method in the Microsoft Store submission API to delete an existing package flight submission.

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request body.

Method	Request URI
DELETE	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationid}/flights/{flightId}/submissions/{submissionId}</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <i><token></i> .

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight submission you want to delete. For more information about the Store ID, see View app identity details .
flightId	string	Required. The ID of the package flight that contains the submission to delete. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.
submissionId	string	Required. The ID of the submission to delete. This ID is available in the response data for requests to create a package flight submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to delete a submission for a package flight.

JSON

```
DELETE https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243649 HTTP/1.1
Authorization: Bearer <your access token>
```

Response

If successful, this method returns an empty response body.

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
400	The request parameters are invalid.
404	The specified submission could not be found.
409	The specified submission was found but it could not be deleted in its current state, or the app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Create and manage submissions using Microsoft Store services](#)
- [Manage package flight submissions](#)
- [Get a package flight submission](#)
- [Create a package flight submission](#)
- [Commit a package flight submission](#)
- [Update a package flight submission](#)
- [Get the status of a package flight submission](#)

Get rollout info for a flight submission

Article • 11/01/2022

Use this method in the Microsoft Store submission API to get [package rollout](#) info for a package flight submission. For more information about the process of creating a package flight submission by using the Microsoft Store submission API, see [Manage package flight submissions](#).

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
- Create a package flight submission for one of your apps. You can do this in Partner Center, or you can do this by using the [create a package flight submission](#) method.

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request parameters.

Method	Request URI
GET	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/packagerollout</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token>.

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight submission with the package rollout info you want to get. For more information about the Store ID, see View app identity details .
flightId	string	Required. The ID of the package flight that contains the submission with the package rollout info you want to get. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center .
submissionId	string	Required. The ID of the submission with the package rollout info to get. This ID is available in the response data for requests to create a package flight submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to get the package rollout info for a package flight submission.

JSON
<pre>GET https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243649/packagerollout HTTP/1.1 Authorization: Bearer <your access token></pre>

Response

The following example demonstrates the JSON response body for a successful call to this method for a package flight submission with gradual package rollout enabled. For more details about the values in the response body, see [Package rollout resource](#).

JSON

```
{
  "isPackageRollout": true,
  "packageRolloutPercentage": 25.0,
  "packageRolloutStatus": "PackageRolloutInProgress",
  "fallbackSubmissionId": "1212922684621243058"
}
```

If the package flight submission does not have gradual package rollout enabled, the following response body will be returned.

JSON

```
{
  "isPackageRollout": false,
  "packageRolloutPercentage": 0.0,
  "packageRolloutStatus": "PackageRolloutNotStarted",
  "fallbackSubmissionId": "0"
}
```

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
404	The package flight submission could not be found.
409	The package flight submission does not belong to the specified package flight, or the app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API .

Related topics

- [Gradual package rollout](#)
- [Manage package flight submissions using the Microsoft Store submission API](#)
- [Create and manage submissions using Microsoft Store services](#)

Update the rollout percentage for a flight submission

Article • 11/01/2022

Use this method in the Microsoft Store submission API to [update the rollout percentage](#) for a package flight submission. For more information about the process of creating a package flight submission by using the Microsoft Store submission API, see [Manage package flight submissions](#).

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
- Create a submission for one of your apps. You can do this in Partner Center, or you can do this by using the [create an app submission](#) method.
- Enable a gradual package rollout for the submission. You can do this [in Partner Center](#), or you can do this by [using the Microsoft Store submission API](#).

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request parameters.

Method	Request URI
POST	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/updatepackagerolloutperc</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token>.

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight submission with the package rollout percentage you want to update. For more information about the Store ID, see View app identity details .
flightId	string	Required. The ID of the package flight that contains the submission with the package rollout percentage you want to update. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.
submissionId	string	Required. The ID of the submission with the package rollout percentage you want to update. This ID is available in the response data for requests to create a package flight submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.
percentage	float	Required. The percentage of users who will receive the gradual rollout package.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to update the package rollout percentage for a package flight submission.

JSON
<pre>POST https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243680/updatepackagerolloutpercentage?percentage=25 HTTP/1.1</pre>

Authorization: Bearer <your access token>

Response

The following example demonstrates the JSON response body for a successful call to this method. For more details about the values in the response body, see [Package rollout resource](#).

JSON

```
{
  "isPackageRollout": true,
  "packageRolloutPercentage": 25.0,
  "packageRolloutStatus": "PackageRolloutInProgress",
  "fallbackSubmissionId": "1212922684621243058"
}
```

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
404	The package flight submission could not be found.
409	This code indicates one of the following errors: - The submission is not in a valid state for the gradual rollout operation (before calling this method, the submission must be published and the packageRolloutStatus value must be set to PackageRolloutInProgress). - The submission does not belong to the specified app. - The app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API.

Related topics

- [Gradual package rollout](#)
- [Manage package flight submissions using the Microsoft Store submission API](#)
- [Create and manage submissions using Microsoft Store services](#)

Halt the rollout for a flight

Article • 11/01/2022

Use this method in the Microsoft Store submission API to [halt the rollout](#) for a package flight submission. For more information about the process of creating a package flight submission by using the Microsoft Store submission API, see [Manage package flight submissions](#).

Note

If you halt the rollout for a package flight submission and then [create a new package flight submission](#), the new submission is a clone of the halted submission.

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
- Create a submission for one of your apps. You can do this in Partner Center, or you can do this by using the [create an app submission](#) method.
- Enable a gradual package rollout for the submission. You can do this [in Partner Center](#), or you can do this by [using the Microsoft Store submission API](#).

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request parameters.

Method	Request URI
POST	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/haltpackagerollout</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <token> .

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight submission with the package rollout you want to halt. For more information about the Store ID, see View app identity details .
flightId	string	Required. The ID of the package flight that contains the submission with the package rollout you want to halt. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.
submissionId	string	Required. The ID of the submission with the package rollout to halt. This ID is available in the response data for requests to create a package flight submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to halt the package rollout for a package flight submission.

```
JSON

POST https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243680/haltpackagerollout HTTP/1.1
Authorization: Bearer <your access token>
```

Response

The following example demonstrates the JSON response body for a successful call to this method. For more details about the values in the response body, see [Package rollout resource](#).

```
JSON

{
  "isPackageRollout": true,
  "packageRolloutPercentage": 0.0,
  "packageRolloutStatus": "PackageRolloutStopped",
  "fallbackSubmissionId": "1212922684621243058"
}
```

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
404	The package flight submission could not be found.
409	This code indicates one of the following errors: - The submission is not in a valid state for the gradual rollout operation (before calling this method, the submission must be published and the <code>packageRolloutStatus</code> value must be set to <code>PackageRolloutInProgress</code>). - The submission does not belong to the specified app. - The app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API.

Related topics

- [Gradual package rollout](#)
- [Manage package flight submissions using the Microsoft Store submission API](#)
- [Create and manage submissions using Microsoft Store services](#)

Finalize the rollout for a flight submission

Article • 11/01/2022

Use this method in the Microsoft Store submission API to [finalize the package rollout](#) for a package flight submission. For more information about the process of creating a package flight submission by using the Microsoft Store submission API, see [Manage package flight submissions](#).

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store submission API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.
- Create a submission for an app in Partner Center. You can do this in Partner Center, or you can do this by using the [create an app submission](#) method.
- Enable a gradual package rollout for the submission. You can do this in [Partner Center](#), or you can do this by [using the Microsoft Store submission API](#).

Request

This method has the following syntax. See the following sections for usage examples and descriptions of the header and request parameters.

Method	Request URI
POST	<code>https://manage.devcenter.microsoft.com/v1.0/my/applications/{applicationId}/flights/{flightId}/submissions/{submissionId}/finalizepackagerollout</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <i><token></i> .

Request parameters

Name	Type	Description
applicationId	string	Required. The Store ID of the app that contains the package flight submission with the package rollout you want to finalize. For more information about the Store ID, see View app identity details .
flightId	string	Required. The ID of the package flight that contains the submission with the package rollout you want to finalize. This ID is available in the response data for requests to create a package flight and get package flights for an app . For a flight that was created in Partner Center, this ID is also available in the URL for the flight page in Partner Center.
submissionId	string	Required. The ID of the submission with the package rollout you want to finalize. This ID is available in the response data for requests to create a package flight submission . For a submission that was created in Partner Center, this ID is also available in the URL for the submission page in Partner Center.

Request body

Do not provide a request body for this method.

Request example

The following example demonstrates how to finalize the package rollout for a package flight submission.

JSON
<pre>POST https://manage.devcenter.microsoft.com/v1.0/my/applications/9NBLGGH4R315/flights/43e448df-97c9-4a43-a0bc-2a445e736bcd/submissions/1152921504621243680/finalizepackagerollout HTTP/1.1 Authorization: Bearer <your access token></pre>

Response

The following example demonstrates the JSON response body for a successful call to this method. For more details about the values in the response body, see [Package rollout resource](#).

JSON

```
{
  "isPackageRollout": true,
  "packageRolloutPercentage": 100.0,
  "packageRolloutStatus": "PackageRolloutComplete",
  "fallbackSubmissionId": "1212922684621243058"
}
```

Error codes

If the request cannot be successfully completed, the response will contain one of the following HTTP error codes.

Error code	Description
404	The package flight submission could not be found.
409	This code indicates one of the following errors: - The submission is not in a valid state for the gradual rollout operation (before calling this method, the submission must be published and the packageRolloutStatus value must be set to <code>PackageRolloutInProgress</code>). - The submission does not belong to the specified app. - The app uses a Partner Center feature that is currently not supported by the Microsoft Store submission API.

Related topics

- [Gradual package rollout](#)
- [Manage package flight submissions using the Microsoft Store submission API](#)
- [Create and manage submissions using Microsoft Store services](#)

Code examples for the submission API

Article • 10/20/2022

This section provides code examples for using the [Microsoft Store submission API](#) in several different programming languages.

ⓘ Note

In addition to the code examples listed below, we also provide an open-source PowerShell module which implements a command-line interface on top of the Microsoft Store submission API. This module is called **StoreBroker** [↗](#). You can use this module to manage your app, flight, and add-on submissions from the command line instead of calling the Microsoft Store submission API directly, or you can simply browse the source to see more examples for how to call this API. The StoreBroker module is actively used within Microsoft as the primary way that many first-party applications are submitted to the Store. For more information, see our [StoreBroker page on GitHub](#) [↗](#).

App submissions, add-on submissions, and package flight submissions

The following articles provide code examples for using the submission API to create app submissions, add-on submissions, and package flight submissions.

- [C# sample: submissions for apps, add-ons, and flights](#)
- [Java sample: submissions for apps, add-ons, and flights](#)
- [Python sample: submissions for apps, add-ons, and flights](#)

Game options and trailers

The following articles provide code examples for using the submission API to define game-specific options and to submit video trailers for apps.

- [C# sample: app submission with game options and trailers](#)
- [Java sample: app submission with game options and trailers](#)
- [Python sample: app submission with game options and trailers](#)

Related topics

- [Create and manage submissions using Microsoft Store services](#)

C# sample: submissions for apps, add-ons, and flights

Article • 01/04/2023

This article provides C# code examples that demonstrate how to use the [Microsoft Store submission API](#) for these tasks:

- [Create an app submission](#)
- [Create an add-on submission](#)
- [Update an add-on submission](#)
- [Create a package flight submission](#)

You can review each example to learn more about the task it demonstrates, or you can build all the code examples in this article into a console application. To build the examples, create a C# console application named **DeveloperApiCSharpSample** in Visual Studio, copy each example to a separate code file in the project, and build the project.

Prerequisites

These examples use the following libraries:

- Microsoft.WindowsAzure.Storage.dll. This library is available in the [Azure SDK for .NET](#), or you can obtain it by installing the [WindowsAzure.Storage NuGet package](#).
- [Newtonsoft.Json](#) NuGet package from Newtonsoft.

Main program

The following example implements a command line program that calls the other example methods in this article to demonstrate different ways to use the Microsoft Store submission API. To adapt this program for your own use:

- Assign the `ApplicationId`, `InAppProductId`, and `FlightId` properties to the ID of the app, add-on, and package flight you want to manage.
- Assign the `ClientId` and `ClientSecret` properties to the client ID and key for your app, and replace the `tenantid` string in the `TokenEndpoint` URL with the tenant ID for your app. For more information, see [How to associate an Azure AD application with your Partner Center account](#)

```

namespace DeveloperApiCSharpSample
{
    class Program
    {
        static void Main(string[] args)
        {
            var config = new ClientConfiguration()
            {
                ApplicationId = "...",
                InAppProductId = "...",
                FlightId = "...",
                ClientId = "...",
                ClientSecret = "...",
                ServiceUrl = "https://manage.devcenter.microsoft.com",
                TokenEndpoint =
                    "https://login.microsoftonline.com/<tenantid>/oauth2/token",
                Scope = "https://manage.devcenter.microsoft.com",
            };

            new
            FlightSubmissionUpdateSample(config).RunFlightSubmissionUpdateSample();

            new
            InAppProductSubmissionUpdateSample(config).RunInAppProductSubmissionUpdateSa
            mple();

            new
            InAppProductSubmissionCreateSample(config).RunInAppProductSubmissionCreateSa
            mple();

            new
            AppSubmissionUpdateSample(config).RunAppSubmissionUpdateSample();
        }
    }
}

```

ClientConfiguration helper class

The sample app uses the `ClientConfiguration` helper class to pass Azure Active Directory data and app data to each of the example methods that use the Microsoft Store submission API.

C#

```

namespace DeveloperApiCSharpSample
{
    /// <summary>
    /// Configuration class
    /// </summary>
    public class ClientConfiguration
    {
        /// <summary>
        /// Client Id of your AAD app.
    }
}

```

```

    /// Example" ba3c223b-03ab-4a44-aa32-38aa10c27e32
    /// </summary>
    public string ClientId { get; set; }

    /// <summary>
    /// Client secret of your AAD app
    /// </summary>
    public string ClientSecret { get; set; }

    /// <summary>
    /// Service root endpoint.
    /// Example: https://manage.devcenter.microsoft.com
    /// </summary>
    public string ServiceUrl { get; set; }

    /// <summary>
    /// Token endpoint to which the request is to be made. Specific to
    your AAD app
    /// Example: https://login.windows.net/d454d300-128e-2d81-334a-
    27d9b2baf002/oauth2/token
    /// </summary>
    public string TokenEndpoint { get; set; }

    /// <summary>
    /// Resource scope. If not provided (set to null), default one is
    used for the production API
    /// endpoint ("https://manage.devcenter.microsoft.com")
    /// </summary>
    public string Scope { get; set; }

    /// <summary>
    /// Application ID.
    /// Example: 9WZANCRD4AMD
    /// </summary>
    public string ApplicationId { get; set; }

    /// <summary>
    /// In-app-product ID;
    /// Example: 9WZBMAAD4VVV
    /// </summary>
    public string InAppProductId { get; set; }

    /// <summary>
    /// Flight Id
    /// Example: 62211033-c2fa-3934-9b03-d72a6b2a171d
    /// </summary>
    public string FlightId { get; set; }
}
}

```

Create an app submission

The following example implements a class that uses several methods in the Microsoft Store submission API to update an app submission. The `RunAppSubmissionUpdateSample` method in the class creates a new submission as a clone of the last published submission, and then it updates and commits the cloned submission to Partner Center. Specifically, the `RunAppSubmissionUpdateSample` method performs these tasks:

1. To begin, the method [gets data for the specified app](#).
2. Next, it [deletes the pending submission for the app](#), if one exists.
3. It then [creates a new submission for the app](#) (the new submission is a copy of the last published submission).
4. It changes some details for the new submission and upload a new package for the submission to Azure Blob Storage.
5. Next, it [updates](#) and then [commits](#) the new submission to Partner Center.
6. Finally, it periodically [checks the status of the new submission](#) until the submission is successfully committed.

C#

```
namespace DeveloperApiCSharpSample
{
    using System;
    using System.Collections.Generic;
    using System.Globalization;
    using System.Net.Http;
    using System.Threading.Tasks;
    using Newtonsoft.Json.Linq;

    /// <summary>
    /// This sample update does a full submission update, updating listings
    info, images, and packages
    /// </summary>
    public class AppSubmissionUpdateSample
    {
        private ClientConfiguration ClientConfig;

        /// <summary>
        /// Constructor
        /// </summary>
        /// <param name="c">An instance of ClientConfiguration that contains
        all parameters populated</param>
        public AppSubmissionUpdateSample(ClientConfiguration c)
        {
            this.ClientConfig = c;
        }

        public void RunAppSubmissionUpdateSample()
        {
            // *****
            // SETTINGS
        }
    }
}
```

```

// *****
var appId = this.ClientConfig.ApplicationId;
var clientId = this.ClientConfig.ClientId;
var clientSecret = this.ClientConfig.ClientSecret;
var serviceEndpoint = this.ClientConfig.ServiceUrl;
var tokenEndpoint = this.ClientConfig.TokenEndpoint;

// Get authorization token.
Console.WriteLine("Getting authorization token ");
var accessToken =
IngestionClient.GetClientCredentialAccessToken(
    tokenEndpoint,
    clientId,
    clientSecret).Result;

Console.WriteLine("Getting application ");
var client = new IngestionClient(accessToken, serviceEndpoint);
dynamic app = client.Invoke<dynamic>(
    HttpMethod.Get,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.GetApplicationUrlTemplate,
        IngestionClient.Version,
        IngestionClient.Tenant,
        appId),
    requestContent: null).Result;
Console.WriteLine(app.ToString());

// Let's get the last published submission, and print its
contents, just for information.
if (app.lastPublishedApplicationSubmission == null)
{
    // It is not possible to create the very first submission
through the API.
    throw new InvalidOperationException(
        "You need at least one published submission to create
new submissions through API.");
}

// Let's see if there is a pending submission. Warning! If it
was created through the API,
// it will be deleted so that we could create a new one in its
stead.
if (app.pendingApplicationSubmission != null)
{
    var submissionId = app.pendingApplicationSubmission.id.Value
as string;

    // Try deleting it. If it was NOT created via the API, then
you need to manually
    // delete it from the dashboard. This is done as a safety
measure to make sure that a
    // user and an automated system don't make conflicting
edits.

    Console.WriteLine("Deleting the pending submission");
}

```

```

        client.Invoke<dynamic>(
            HttpMethod.Delete,
            relativeUrl: string.Format(
                CultureInfo.InvariantCulture,
                IngestionClient.GetSubmissionUrlTemplate,
                IngestionClient.Version,
                IngestionClient.Tenant,
                appId,
                submissionId),
            requestContent: null).Wait();
    }

    // Create a new submission, which will be an exact copy of the
    last published submission.
    Console.WriteLine("Creating a new submission");
    dynamic clonedSubmission = client.Invoke<dynamic>(
        HttpMethod.Post,
        relativeUrl: string.Format(
            CultureInfo.InvariantCulture,
            IngestionClient.CreateSubmissionUrlTemplate,
            IngestionClient.Version,
            IngestionClient.Tenant,
            appId),
        requestContent: null).Result;

    // Update some property on the root submission object.
    clonedSubmission.notesForCertification = "This is a test update,
    updating listing info, images, and packages";

    // Now, assume we have an en-us listing. Let's try to change its
    description.
    clonedSubmission.listings["en-us"].baseListing.description =
    "This is my new en-US description!";

    // Update images.
    // Assuming we have at least 1 image, let's delete one image.
    clonedSubmission.listings["en-
    us"].baseListing.images[0].fileStatus = "PendingDelete";

    var images = new List<dynamic>();
    images.Add(clonedSubmission.listings["en-
    us"].baseListing.images[0]);
    images.Add(
        new
        {
            fileStatus = "PendingUpload",
            fileName = "rectangles.png",
            imageType = "Screenshot",
            description = "This is a new image uploaded through the
    API!",
        });

    clonedSubmission.listings["en-us"].baseListing.images =
    JToken.FromObject(images.ToArray());

```



```

        // Update packages.
        // Let's say we want to delete the existing package.
        clonedSubmission.applicationPackages[0].fileStatus =
"PendingDelete";

        // Now, let's add a new package.
        var packages = new List<dynamic>();
        packages.Add(clonedSubmission.applicationPackages[0]);
        packages.Add(
            new
            {
                fileStatus = "PendingUpload",
                fileName = "package.appx",
                minimumDirectXVersion = "None",
                minimumSystemRam = "None"
            });

        clonedSubmission.applicationPackages =
JToken.FromObject(packages.ToArray());
        var clonedSubmissionId = clonedSubmission.id.Value as string;

        // Uploaded the zip archive with all new files to the SAS url
        returned with the submission.
        var fileUploadUrl = clonedSubmission.fileUploadUrl.Value as
string;

        Console.WriteLine("FileUploadUrl: " + fileUploadUrl);
        Console.WriteLine("Uploading file");
        IngestionClient.UploadFileToBlob(@"..\..\files.zip",
fileUploadUrl).Wait();

        // Update the submission.
        Console.WriteLine("Updating the submission");
        client.Invoke<dynamic>(
            HttpMethod.Put,
            relativeUrl: string.Format(
                CultureInfo.InvariantCulture,
                IngestionClient.UpdateUrlTemplate,
                IngestionClient.Version,
                IngestionClient.Tenant,
                appId,
                clonedSubmissionId),
            requestContent: clonedSubmission).Wait();

        // Tell the system that we are done updating the submission.
        // Update the submission.
        Console.WriteLine("Committing the submission");
        client.Invoke<dynamic>(
            HttpMethod.Post,
            relativeUrl: string.Format(
                CultureInfo.InvariantCulture,
                IngestionClient.CommitSubmissionUrlTemplate,
                IngestionClient.Version,
                IngestionClient.Tenant,
                appId,

```

```

        clonedSubmissionId),
        requestContent: null).Wait();

    // Let's periodically check the status until it changes from
    "CommitsStarted" to either
    // successful status or a failure.
    Console.WriteLine("Waiting for the submission commit processing
    to complete. This may take a couple of minutes.");
    string submissionStatus = null;
    do
    {
        Task.Delay(TimeSpan.FromSeconds(5)).Wait();
        dynamic statusResource = client.Invoke<dynamic>(
            HttpMethod.Get,
            relativeUrl: string.Format(
                CultureInfo.InvariantCulture,
                IngestionClient.ApplicationSubmissionStatusUrlTemplate,
                IngestionClient.Version,
                IngestionClient.Tenant,
                appId,
                clonedSubmissionId),
            requestContent: null).Result;

        submissionStatus = statusResource.status.Value as string;
        Console.WriteLine("Current status: " + submissionStatus);
    }
    while ("CommitStarted".Equals(submissionStatus));

    if ("CommitFailed".Equals(submissionStatus))
    {
        Console.WriteLine("Submission has failed. Please checkt the
        Errors collection of the submissionResource response.");
        return;
    }
    else
    {
        Console.WriteLine("Submission commit success! Here are some
        data:");
        dynamic submission = client.Invoke<dynamic>(
            HttpMethod.Get,
            relativeUrl: string.Format(
                CultureInfo.InvariantCulture,
                IngestionClient.GetSubmissionUrlTemplate,
                IngestionClient.Version,
                IngestionClient.Tenant,
                appId,
                clonedSubmissionId),
            requestContent: null).Result;
        Console.WriteLine("Packages: " +
        submission.applicationPackages);
        Console.WriteLine("en-US description: " +
        submission.listings["en-us"].baseListing.description);
        Console.WriteLine("Images: " + submission.listings["en-
        us"].baseListing.images);
    }
}

```

```
}  
}  
}  
}
```

Create an add-on submission

The following example implements a class that uses several methods in the Microsoft Store submission API to create a new add-on submission. The

`RunInAppProductSubmissionCreateSample` method in the class performs these tasks:

1. To begin, the method [creates a new add-on](#).
2. Next, it [creates a new submission for the add-on](#).
3. It uploads a ZIP archive that contains icons for the submission to Azure Blob Storage.
4. Next, it [commits the new submission to Partner Center](#).
5. Finally, it periodically [checks the status of the new submission](#) until the submission is successfully committed.

C#

```
namespace DeveloperApiCSharpSample  
{  
    using System;  
    using System.Collections.Generic;  
    using System.Globalization;  
    using System.Net.Http;  
    using System.Threading.Tasks;  
  
    /// <summary>  
    /// Sample code for how to create add-ons, and how to create and update  
    /// add-on submissions.  
    /// </summary>  
    public class InAppProductSubmissionCreateSample  
    {  
        private ClientConfiguration ClientConfig;  
  
        /// <summary>  
        /// Constructor  
        /// </summary>  
        /// <param name="c">An instance of ClientConfiguration that contains  
        all parameters populated</param>  
        public InAppProductSubmissionCreateSample(ClientConfiguration c)  
        {  
            this.ClientConfig = c;  
        }  
  
        public void RunInAppProductSubmissionCreateSample()
```

```

{
    // *****
    //     SETTINGS
    // *****
    var appId = this.ClientConfig.ApplicationId;
    var clientId = this.ClientConfig.ClientId;
    var clientSecret = this.ClientConfig.ClientSecret;
    var serviceEndpoint = this.ClientConfig.ServiceUrl;
    var tokenEndpoint = this.ClientConfig.TokenEndpoint;

    // Get authorization token
    Console.WriteLine("Getting authorization token ");
    var accessToken =
IngestionClient.GetClientCredentialAccessToken(
    tokenEndpoint,
    clientId,
    clientSecret).Result;

    Console.WriteLine("Creating a new add-on");
    dynamic newIap = new
    {
        applicationIds = new List<string>() { appId },
        productType = "Durable",
        productId = "Sample-" + Guid.NewGuid().ToString(),
    };

    var client = new IngestionClient(accessToken, serviceEndpoint);
    dynamic iapCreated = client.Invoke<dynamic>(
        HttpMethod.Post,
        relativeUrl: string.Format(
            CultureInfo.InvariantCulture,
            IngestionClient.CreateInAppUrlTemplate,
            IngestionClient.Version,
            IngestionClient.Tenant),
        requestContent: newIap).Result;
    Console.WriteLine(iapCreated.ToString());
    var iapId = iapCreated.id.Value as string;

    // Create a new submission, which will be an exact copy of the
    last published submission
    Console.WriteLine("Creating a new submission");
    dynamic newSubmission = new
    {
        contentType = "BookDownload",
        keywords = new List<string> { "book", "download" },
        lifeTime = "ThreeDays",
        targetPublishMode = "Immediate",
        visibility = "Public",
        pricing = new
        {
            priceId = "Free",
        },
        listings = new Dictionary<string, dynamic>()
        {
            {

```

```

        "en-us",
        new
        {
            description = "Sample IAP description",
            title = "Sample IAP title",
            icon = new
            {
                FileName = "icon300x300.png",
                FileStatus = "PendingUpload",
            },
        }
    }
}
};

```

```

// Because it's a new add-on, we are going to create a new
submission instead of
// modifying the last published one. If you had a published add-
on, you could
// pass "null" as request body to clone the latest published
submission and then
// perform a PUT call. Alternatively, you can always post the
new submission entirely
// even if you already have a published submission but you'll
have to upload the image each time.

```

```

dynamic createdSubmission = client.Invoke<dynamic>(
    HttpMethod.Post,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.InAppSubmissionUrl,
        IngestionClient.Version,
        IngestionClient.Tenant,
        iapId),
    requestContent: newSubmission).Result;
Console.WriteLine(createdSubmission);
var submissionId = createdSubmission.id.Value as string;

// Upload the zip archive with all new files to the SAS URL
returned with the submission.
var fileUploadUrl = createdSubmission.fileUploadUrl.Value as
string;

Console.WriteLine("FileUploadUrl: " + fileUploadUrl);
Console.WriteLine("Uploading file");
IngestionClient.UploadFileToBlob(@"..\..\files.zip",
fileUploadUrl).Wait();

```

```

// Tell the system that we are done updating the submission.
// Update the submission
Console.WriteLine("Committing the submission");
client.Invoke<dynamic>(
    HttpMethod.Post,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.InAppProductCommitSubmissionUrlTemplate,
        IngestionClient.Version,

```